perplexity

So now will focus on this in deep and tell me what we will provide to user like what platform, library, or what you tell?

And also this project should be impactfull and compared to other technology they are used they should be benchmarked yes can this be a milestone

Yes—**AgentBudget should be both a Python library and a lightweight control-plane platform**, not merely an observability dashboard. Its unique promise is: developers define a task contract once, and AgentBudget enforces cost, latency, reliability, and safety limits while the agent runs. That is different from frameworks such as LangGraph, CrewAI, AutoGen, and OpenAI Agents SDK, which primarily provide orchestration, state, handoffs, and tool-calling rather than task-level budget enforcement.[1]

What users get

You should provide **four layers**, so it is useful immediately but can grow into a platform.

Layer	What the user receives	Example
Python SDK	Decorators, context manager, budget object, tool wrapper	`@agent_task(max_cost_usd=0.50)`
Runtime harness	Agent loop, tool dispatch, retries, stop rules, context management	Built-in reliable ReAct-style executor
Control plane	Dashboard, traces, budgets, alerts, policy management	"This task spent $0.31 and passed SLO"
Integrations	Works with existing agent frameworks and vLLM/OpenAI-compatible models	LangGraph, CrewAI, OpenAI Agents SDK adapters

The SDK is the adoption path; the platform is the retention and enterprise path.

Developer experience

A user should be able to start with something this simple:

```
from agentbudget import AgentTask, tool

@tool
def search_docs(query: str) -> str:
    """Search internal documentation."""
    ...
```

```
@tool
def run_tests() -> dict:
    """Run the project test suite."""
    ...

task = AgentTask(
    goal="Find duplicated pricing logic and make the tests pass.",
    max_llm_calls=12,
    max_tool_calls=15,
    max_tokens=60_000,
    max_cost_usd=0.50,
    max_wall_time_seconds=300,
    success_criteria=[
        "pytest exits successfully",
        "public API behavior is unchanged",
    ],
)

result = task.run(agent="coding-agent")
print(result.receipt)
```

AgentBudget then controls the loop, records every step, prevents runaway behavior, and returns a **Task Receipt**.

The Task Receipt

This is your flagship user-facing artifact.

```
{
  "task_id": "task_8f21",
  "status": "completed",
  "success": true,
  "tokens": 41200,
  "cost_usd": 0.29,
  "llm_calls": 9,
  "tool_calls": 6,
  "retries": 1,
  "wall_time_seconds": 94,
  "duplicate_tool_calls_prevented": 3,
  "context_tokens_saved": 21000,
  "slo_met": true,
  "waste_report": [
    {
      "type": "duplicate_file_read",
      "saved_tokens": 8400,
      "saved_cost_usd": 0.021
    }
  ]
}
```

Most observability tools can tell you what a run cost after the fact. AgentBudget tells you **during the run** whether the task is still worth continuing, and it can stop, compress, switch strategy, or escalate. OpenTelemetry GenAI conventions already standardize model calls, token usage, tool

calls, and latency, so AgentBudget should emit compatible traces rather than inventing a proprietary telemetry format. [2] [3] [4]

What the platform provides

1. Budget policies

Teams can define reusable policies:

```
policy: coding-agent-production
limits:
  max_cost_usd: 1.00
  max_tokens: 150000
  max_tool_calls: 30
  max_retries: 3
  max_wall_time_seconds: 600
quality:
  required_checks:
    - tests_pass
    - no_unapproved_file_write
escalation:
  on_budget_warning: notify
  on_budget_exceeded: stop_and_summarize
  on_repeated_failure: human_review
```

2. Live control room

A dashboard showing:

- Active agent tasks
- Cost and token burn in real time
- Remaining budget
- Tool-call timeline
- Loop and duplicate-call alerts
- Retry and failure reasons
- Model/model-route distribution
- p95 latency
- Success rate by task type
- Cost per successful task

The most important metric is **cost per successful task**, not cost per token. Agent-level tracing should tie every step to a run/session ID and report cost per completed task. [5] [6]

3. Waste analyzer

AgentBudget should automatically classify waste:

- Duplicate tool call
- Repeated failed call
- Oversized tool output
- Irrelevant retrieved context
- Redundant model call
- Retry storm
- Agent loop
- Unnecessary model upgrade
- Context that could have been summarized

Then it recommends or applies a fix.

4. Replay and regression testing

Users can replay a failed task with:

- The same tools
- A different model
- A different context policy
- A stricter retry policy
- A new budget

This turns agent failures into reproducible test cases.

How it compares

Capability	LangGraph / CrewAI / OpenAI Agents SDK	Observability tools	AgentBudget
Agent orchestration	Strong	Limited	Good enough for single-agent MVP
Tool calling	Strong	Traces it	Governs and validates it
State and memory	Framework-specific	Records it	Manages it for cost and reliability
Tracing	Varies	Strong	Strong, OpenTelemetry-compatible
Cost tracking	Often basic	Strong after the fact	Enforced during the run
Loop detection	Usually manual	Detects after the fact	Prevents and repairs in real time
Task SLOs	Rare	Reports them	Defines and enforces them

Capability	LangGraph / CrewAI / OpenAI Agents SDK	Observability tools	AgentBudget
Budget stop/escalation	Rare	Rare	Core feature
Framework lock-in	Often high	Low	Low—SDK plus adapters

Existing frameworks are often free as SDKs, but real production cost comes from token usage, managed hosting, observability, and error-handling infrastructure. AgentBudget attacks that last category directly.[7] [1]

Benchmark strategy

Yes—**benchmarking is essential**, and it can become a milestone if you publish a reproducible benchmark, not just marketing claims.

Benchmark name

AgentBudgetBench

Or, more ambitious:

AgentEfficiencyBench (AEB)

What it measures

Do not benchmark only accuracy. Measure the full production equation:

Metric	Definition
Task success rate	Percentage of tasks meeting declared success criteria
Cost per successful task	Total spend divided by successful tasks
Token efficiency	Tokens used per successful task
Tool efficiency	Useful tool calls divided by total tool calls
Retry rate	Retries divided by total tool calls
Loop rate	Tasks with repeated near-identical actions
p95 latency	95th-percentile task completion time
Budget compliance	Percentage of tasks finishing within declared limits
Quality retention	Quality relative to an unconstrained baseline
Escalation rate	Percentage of tasks requiring human help

Agent reliability benchmarks such as τ-bench use pass@k-style evaluation, and published agent evaluations already show that reliability can fall sharply across repeated attempts; that makes reliability-plus-efficiency benchmarking especially relevant.[8]

Benchmark tasks

Start with four reproducible task families:

1. **Code repair:** fix a bug and make hidden tests pass.
2. **Repository analysis:** find duplicated logic and propose a refactor.
3. **RAG research:** answer a question using a fixed document set.
4. **Tool workflow:** complete a multi-step API workflow with rate limits and failures injected.

For each task, compare:

```
Baseline agent
vs
Baseline agent + tracing
vs
AgentBudget harness
vs
AgentBudget + model routing
```

Report a table like:

System	Success rate	Cost/task	Tokens/task	Tool calls	p95 latency	SLO compliance
Baseline	78%	$1.24	168k	18	310s	61%
AgentBudget	82%	$0.41	52k	7	140s	94%

That kind of result would be genuinely compelling.

Can it be a milestone?

Yes—if you define the milestone carefully.

Milestone 1: Working harness

- Python SDK
- Budget enforcement
- Tool registry
- Retry and stop rules
- Task receipts
- Local dashboard

Milestone 2: Open benchmark

- Public dataset
- Deterministic tasks
- Baselines

- Cost/quality/latency reporting
- One-command reproduction

Milestone 3: Framework adapters

- LangGraph adapter
- CrewAI adapter
- OpenAI Agents SDK adapter
- OpenTelemetry export

Milestone 4: Policy engine

- YAML/JSON budget policies
- Team-level quotas
- Escalation workflows
- Model-routing integration

Milestone 5: Production control plane

- Hosted or self-hosted dashboard
- Alerts
- Policy versioning
- Cost attribution by team, agent, task type, and model

Honest differentiation

Do **not** claim AgentBudget replaces LangGraph, CrewAI, AutoGen, or OpenAI Agents SDK. Position it as the missing **reliability and economics layer**:

> **"Agent frameworks help you build agents. AgentBudget helps you run them safely, affordably, and predictably in production."**

That is a strong, defensible position. The opportunity is real because agent failures often come from harness-level issues—tool errors, loops, bad retries, weak stop conditions, and poor context management—while most current tooling focuses on building agents or observing them after the fact.[9] [10] [11]

Your first public milestone should be:

> **AgentBudget v0.1: A budget-native Python agent harness with task receipts, loop prevention, tool governance, and reproducible AgentBudgetBench results.**

If you ship that with a clear demo and honest benchmark, it can become a meaningful open-source project—not just another agent framework.[1] [5] [8]

⁂

1. https://evermind.ai/blogs/ai-agent-orchestration-frameworks-compared
2. https://genkit.dev/docs/observability/otel-genai-semantic-conventions/
3. https://opentelemetry.io/docs/specs/semconv/registry/attributes/gen-ai/
4. https://nango.dev/blog/opentelemetry-ai-agent-observability/
5. https://www.nops.io/blog/ai-cost-observability/
6. https://clickhouse.com/resources/engineering/what-is-ai-agent-observability
7. https://www.langchain.com/resources/ai-agent-frameworks
8. https://arxiv.org/html/2511.14136v1
9. https://www.fiddler.ai/blog/building-reliable-agents-takes-more-than-better-models
10. https://atlan.com/know/agent-harness-failures-anti-patterns/
11. https://www.openlayer.com/blog/ai-agent-failure-modes-tool-calling-loops-propagation
12. https://genkit.dev/docs/dart/observability/otel-genai-semantic-conventions/
13. https://pkg.go.dev/github.com/plexusone/omniobserve/semconv/agent
14. https://greptime.com/blogs/2026-05-09-opentelemetry-genai-semantic-conventions
15. https://kanerika.com/blogs/ai-agent-frameworks/
16. https://github.com/open-telemetry/opentelemetry.io/blob/main/content/en/blog/2026/genai-observability/index.md
17. https://github.com/open-telemetry/semantic-conventions-genai/issues/35
18. https://github.com/howells/agentsurface/blob/main/src/content/docs/testing/observability.mdx
19. https://github.com/Cloudgeni-ai/infrastructure-agents-guide/blob/main/09-observability.md